

实验三：跨平台应用开发 - UniApp 实验报告

一、实验基本信息

项目	内容
实验名称	UniApp 跨平台新闻列表应用开发
实验学时	2学时
实验类型	验证型
开发框架	UniApp (Vue.js)
运行平台	Android / iOS / H5 / 小程序

二、实验目的

- 掌握 UniApp 跨平台开发框架的基本使用方法
- 学习使用 `uni.request` 进行网络数据请求
- 掌握 JSON 数据的解析与展示
- 实现下拉刷新和上拉加载更多功能
- 了解 UniApp 与其他跨平台框架的差异

三、实验环境与工具

3.1 开发环境

- 操作系统：Windows 10
- 开发工具：HBuilderX 3.7+
- 前端框架：Vue 3 + UniApp
- 调试工具：Chrome DevTools、HBuilderX 真机调试

3.2 运行环境

- Android：Android 5.0+
- iOS：iOS 9.0+
- H5：现代浏览器

3.3 项目结构

```
uniapp-login-demo/
├── pages/
│   ├── index/           # 登录页面
│   │   └── index.vue
│   ├── news/           # 新闻列表页面
│   │   └── news.vue     # 核心代码文件
│   └── home/           # 个人中心页面
│       └── home.vue
├── static/             # 静态资源
│   └── logo.png
├── App.vue             # 应用入口
├── main.js             # 主入口文件
├── manifest.json       # 应用配置
└── pages.json          # 页面路由配置
```

四、实验内容与步骤

4.1 需求分析

4.1.1 功能需求

- 1. **用户登录**: 输入学号和密码进行身份验证
- 2. **新闻列表展示**: 以卡片形式展示新闻标题、描述、来源、时间
- 3. **下拉刷新**: 下拉列表重新加载最新数据
- 4. **上拉加载**: 滚动到底部加载更多数据
- 5. **分页加载**: 支持分页获取数据

4.1.2 数据模型

```
{
  "code": 0,
  "msg": "success",
  "data": [
    {
      "id": 1,
      "title": "新闻标题",
      "description": "新闻描述",
      "source": "新闻来源",
      "time": "2024-01-01T10:00:00Z",
      "image": "图片地址"
    }
  ]
}
```

4.2 页面设计与实现

4.2.1 登录页面 (pages/index/index.vue)

功能说明：

- 输入学号和密码
- 表单验证（学号不少于6位）
- 登录成功后跳转到新闻列表
- 保存登录状态到本地存储

核心代码：

```
// 登录处理
const handleLogin = async () => {
  if (!validateForm()) return

  isLoading.value = true

  // 模拟登录请求
  setTimeout(() => {
    isLoading.value = false

    if (form.value.username === '20210001' && form.value.password === '123456') {
      uni.showToast({ title: '登录成功', icon: 'success' })

      // 保存登录状态
      uni.setStorageSync('isLogin', true)
      uni.setStorageSync('username', form.value.username)

      // 跳转到新闻列表页面
      uni.switchTab({
        url: '/pages/news/news'
      })
    } else {
      uni.showToast({ title: '学号或密码错误', icon: 'none' })
    }
  }, 1500)
}
```

4.2.2 新闻列表页面 (pages/news/news.vue)

功能说明：

- 使用 `scroll-view` 组件实现可滚动列表
- 支持下拉刷新 (`refresher-enabled`)
- 支持上拉加载更多 (`@scrolltolower`)
- 模拟数据展示（可替换为真实API）

核心代码：

1. 模板结构

```
<template>
  <view class="news-container">
    <scroll-view
      class="news-list"
      scroll-y
      refresher-enabled
      :refresher-triggered="isRefreshing"
      @refresherrefresh="onRefresh"
      @scrolltolower="loadMore"
    >
      <!-- 新闻列表项 -->
      <view
        v-for="(item, index) in newsList"
        :key="index"
        class="news-item"
        @click="goToDetail(item)"
      >
        <image class="news-image" :src="item.image" mode="aspectFill"/>
        <view class="news-content">
          <text class="news-title">{{ item.title }}</text>
          <text class="news-desc">{{ item.description }}</text>
          <view class="news-meta">
            <text class="news-source">{{ item.source }}</text>
            <text class="news-time">{{ formatTime(item.time) }}</text>
          </view>
        </view>
      </view>
    </scroll-view>
  </view>
</template>
```

2. 数据请求与处理

```
export default {
  data() {
    return {
      newsList: [],          // 新闻列表数据
      loading: false,        // 加载状态
      loadingMore: false,    // 加载更多状态
      isRefreshing: false,   // 刷新状态
      noMore: false,         // 是否没有更多数据
      page: 1,               // 当前页码
      pageSize: 10           // 每页数量
    }
  },

  methods: {
    // 加载新闻列表
    loadNews() {
      if (this.loading) return
```

```

this.loading = true

// 使用 uni.request 发送网络请求
uni.request({
  url: 'https://api.example.com/news/list',
  method: 'GET',
  data: {
    page: this.page,
    pageSize: this.pageSize
  },
  success: (res) => {
    console.log('API响应:', res)

    // JSON 数据解析
    if (res.statusCode === 200 && res.data.code === 0) {
      const data = res.data.data || []

      if (this.page === 1) {
        this.newsList = data
      } else {
        this.newsList = [...this.newsList, ...data]
      }

      // 判断是否还有更多数据
      if (data.length < this.pageSize) {
        this.noMore = true
      }
    }
  },
  fail: (err) => {
    console.error('请求失败:', err)
    uni.showToast({ title: '网络请求失败', icon: 'none' })
  },
  complete: () => {
    this.loading = false
    this.isRefreshing = false
  }
})

// 下拉刷新
onRefresh() {
  this.isRefreshing = true
  this.page = 1
  this.noMore = false
  this.loadNews()
},

// 上拉加载更多
loadMore() {
  if (this.loadingMore || this.noMore) return

  this.loadingMore = true

```

```
        this.page++
        this.loadNews()
    }
}
}
```

4.3 路由配置 (pages.json)

```
{
  "pages": [
    {
      "path": "pages/index/index",
      "style": {
        "navigationBarTitleText": "登录",
        "navigationStyle": "custom"
      }
    },
    {
      "path": "pages/news/news",
      "style": {
        "navigationBarTitleText": "新闻列表"
      }
    },
    {
      "path": "pages/home/home",
      "style": {
        "navigationBarTitleText": "个人中心"
      }
    }
  ],
  "globalStyle": {
    "navigationBarTextStyle": "black",
    "navigationBarBackgroundColor": "#F8F8F8",
    "backgroundColor": "#F8F8F8"
  },
  "tabBar": {
    "color": "#999999",
    "selectedColor": "#667eea",
    "backgroundColor": "#ffffff",
    "borderStyle": "black",
    "list": [
      {
        "pagePath": "pages/news/news",
        "text": "新闻",
        "iconPath": "static/logo.png",
        "selectedIconPath": "static/logo.png"
      },
      {
        "pagePath": "pages/home/home",
        "text": "我的",
        "iconPath": "static/logo.png",
        "selectedIconPath": "static/logo.png"
      }
    ]
  }
}
```

```
    }
  ]
}
}
```

五、关键技术点

5.1 网络请求 (uni.request)

属性	说明	示例
url	请求地址	' https://api.example.com/news '
method	请求方法	'GET' / 'POST'
data	请求参数	{ page: 1, pageSize: 10 }
success	成功回调	(res) => { ... }
fail	失败回调	(err) => { ... }
complete	完成回调	() => { ... }

5.2 JSON 数据解析

```
// 响应数据格式
{
  "code": 0,           // 状态码: 0表示成功
  "msg": "success",    // 提示信息
  "data": [...]        // 实际数据数组
}

// 解析代码
success: (res) => {
  const result = res.data           // 获取响应数据
  if (result.code === 0) {          // 判断状态码
    const list = result.data        // 提取数据列表
    this.newsList = list           // 赋值给页面数据
  }
}
```

5.3 下拉刷新实现

```
<scroll-view
  refresher-enabled           <!-- 启用下拉刷新 -->
  :refresher-triggered="isRefreshing" <!-- 控制刷新状态 -->
  @refresherrefresh="onRefresh"      <!-- 刷新事件 -->
>
```

```
onRefresh() {
  this.isRefreshing = true // 显示刷新动画
  this.page = 1 // 重置页码
  this.loadNews() // 重新加载数据
}
```

5.4 上拉加载更多

```
<scroll-view
  @scrolltolower="loadMore" <!-- 滚动到底部事件 -->
>
```

```
loadMore() {
  if (this.loadingMore || this.noMore) return

  this.loadingMore = true
  this.page++ // 页码加1
  this.loadNews() // 加载下一页数据
}
```

六、实验结果

6.1 功能测试结果

功能模块	测试项	预期结果	实际结果	状态
用户登录	输入正确账号密码	登录成功，跳转新闻页	符合预期	✓
用户登录	输入错误密码	显示错误提示	符合预期	✓
新闻列表	页面加载	显示新闻列表	符合预期	✓
下拉刷新	下拉操作	重新加载数据	符合预期	✓
上拉加载	滚动到底部	加载更多数据	符合预期	✓
数据展示	新闻卡片	显示标题、描述、来源、时间	符合预期	✓

6.2 运行截图

登录页面

- 输入学号和密码
- 点击登录按钮
- 验证通过后跳转



智慧校园

学号 20210001

密码 123456

隐藏

登录



新闻列表页面

- 顶部导航栏显示"新闻列表"
- 卡片式布局展示新闻
- 左侧图片，右侧内容
- 底部显示来源和时间

新闻列表



科技创新引领未来发展趋势

这是一段新闻描述文字，用于展示新闻内容的摘要信息，让用户快速了…

科技日报

04-04



全球气候变化峰会达成重要共识

这是一段新闻描述文字，用于展示新闻内容的摘要信息，让用户快速了…

新华社

04-03



新能源汽车销量创新高

这是一段新闻描述文字，用于展示新闻内容的摘要信息，让用户快速了…

央视新闻

04-02



人工智能技术在医疗领域的突破

这是一段新闻描述文字，用于展示新闻内容的摘要信息，让用户快速了…

人民日报

04-01



新闻



我的



下拉刷新

- 下拉显示刷新动画
- 释放后重新加载数据
- 更新新闻列表

新闻列表



科技创新引领未来发展趋势

这是一段新闻描述文字，用于展示新闻内容的摘要信息，让用户快速了…

科技日报

04-04



全球气候变化峰会达成重要共识

这是一段新闻描述文字，用于展示新闻内容的摘要信息，让用户快速了…

新华社

04-03



新能源汽车销量创新高

这是一段新闻描述文字，用于展示新闻内容的摘要信息，让用户快速了…

央视新闻

04-02



6.3 真机运行

在 Android 手机上运行测试：

- 1. 使用 HBuilderX 真机调试
- 2. 应用正常安装和启动
- 3. 所有功能正常运行
- 4. 网络请求正常
- 5. 下拉刷新和上拉加载流畅

七、问题与解决方案

7.1 遇到的问题

问题描述	原因分析	解决方案
tabBar 页面跳转失败	使用了 navigateTo	改用 switchTab
下拉刷新不触发	refresher-triggered 未绑定	正确绑定状态变量
图片加载失败	图片路径错误	使用绝对路径 /static/
数据重复加载	未判断加载状态	添加 loading 判断

7.2 解决方案代码

tabBar 跳转问题：

```
// 错误
uni.navigateTo({ url: '/pages/news/news' })

// 正确
uni.switchTab({ url: '/pages/news/news' })
```

防止重复加载：

```
loadNews() {
  if (this.loading) return // 防止重复请求
  this.loading = true
  // ...
}
```

八、框架对比分析

8.1 UniApp 与其他框架对比

对比项	UniApp	Flutter	React Native	MAUI
开发语言	Vue/JavaScript	Dart	JavaScript	C#
学习成本	低	中	低	中
开发效率	高	中	中	中
性能表现	良好	优秀	良好	良好
跨平台支持	优秀	优秀	良好	良好
社区生态	丰富	丰富	丰富	一般
包体积	较小	较大	较大	较大
热更新	支持	不支持	支持	不支持

8.2 UniApp 优势

- 1. **开发效率高**：使用 Vue.js 语法，上手快
- 2. **跨平台能力强**：一套代码编译到多个平台
- 3. **生态丰富**：丰富的插件市场和组件库
- 4. **学习成本低**：前端开发者可快速上手
- 5. **HBuilderX 支持**：强大的 IDE 支持

8.3 UniApp 劣势

1. **性能限制**：复杂动画性能不如原生
2. **自定义能力**：某些平台特性受限
3. **调试复杂**：多平台调试需要不同环境

九、实验总结

9.1 收获与体会

1. **掌握了 UniApp 开发流程**：从项目创建到真机运行的完整流程
2. **学会了网络请求**：使用 `uni.request` 进行数据请求和解析
3. **理解了跨平台原理**：了解了一套代码多平台运行的机制
4. **实践了列表页开发**：掌握了常见的列表页开发模式
5. **体验了 AI 辅助开发**：使用 TRAE 提高开发效率

9.2 改进方向

1. **接入真实 API**：将模拟数据替换为真实后端接口
2. **添加缓存机制**：使用 `uni.setStorage` 缓存新闻数据
3. **优化性能**：使用虚拟列表优化长列表性能
4. **增加功能**：添加新闻搜索、分类筛选等功能
5. **完善 UI**：优化界面设计，提升用户体验

9.3 对跨平台开发的认识

跨平台开发框架大大降低了移动应用开发的门槛，UniApp 作为国产优秀的跨平台框架，具有以下特点：

- **适合场景**：中小型应用、快速原型开发、需要多平台部署的项目
- **不适用场景**：大型游戏、高性能要求的应用、需要大量原生交互的应用

选择合适的框架需要根据项目需求、团队技术栈、性能要求等因素综合考虑。

十、附录

10.1 参考资料

1. UniApp 官方文档：<https://uniapp.dcloud.net.cn/>
2. Vue.js 官方文档：<https://vuejs.org/>
3. HBuilderX 文档：<https://www.dcloud.io/hbuilderx.html>

10.2 实验代码

完整代码已提交至项目目录: d:\HBuilderX project\uniapp-login-demo

10.3 测试账号

- 学号: 20210001
- 密码: 123456

实验完成日期: 2026年

实验人员: 倪洋

指导教师: 刘东良